Neural Networks - Part II

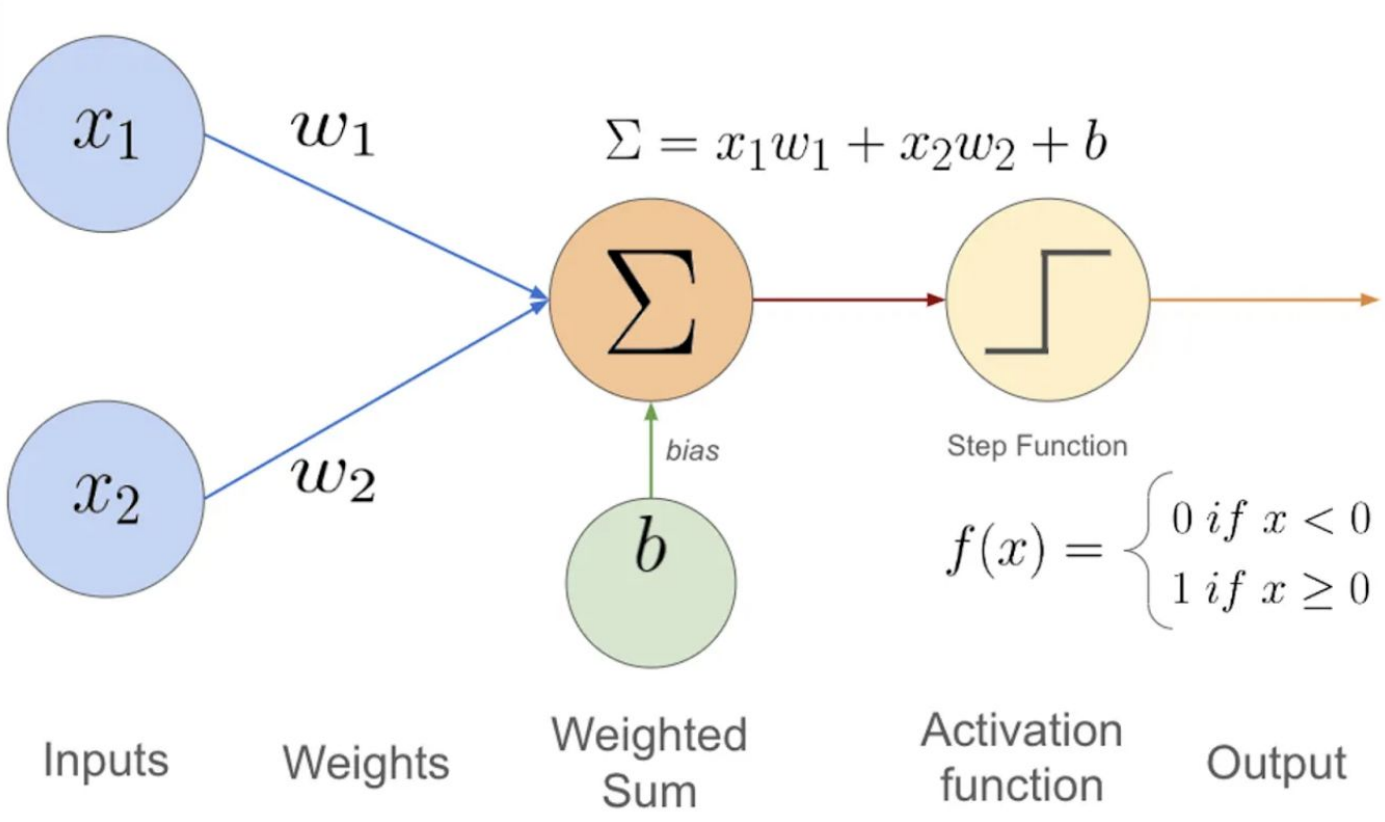
Arash Marioriyad

19 Khordad 1405

CE 417-Artificial Intelligence
Sharif Univ. of Tech.

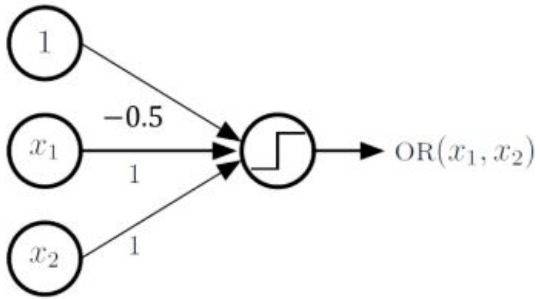
Most Slides have been adopted from Berkeley CS188 & SBU CI course Dr. Malek & SUT CE417 and CE717

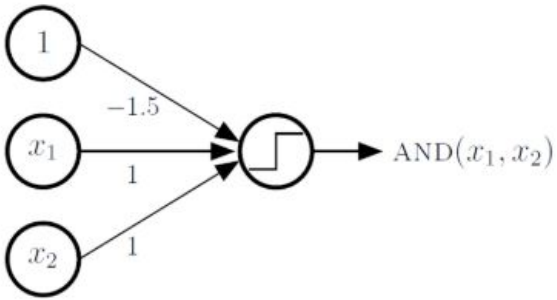
Perceptron


x_1
w_1
$\Sigma = x_1w_1 + x_2w_2 + b$
Σ
x_2
w_2
bias
b
Step Function
$f(x) = \begin{cases} 0 \; if \; x < 0 \\ 1 \; if \; x \geq 0 \end{cases}$
Inputs
Weights
Weighted Sum
Activation function
Output

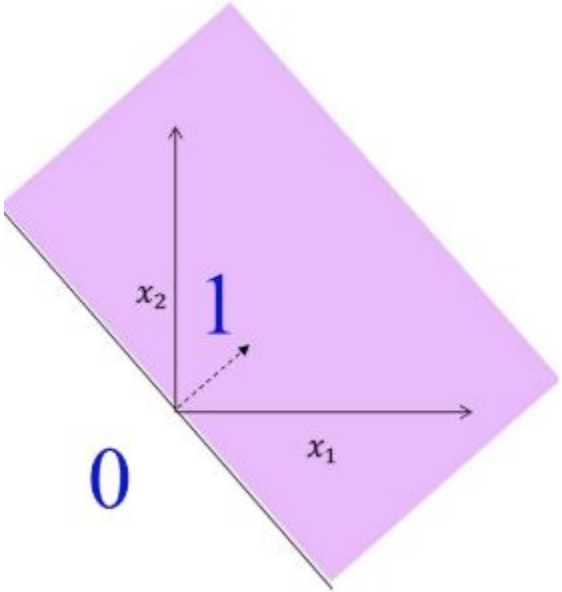
AND & OR Networks

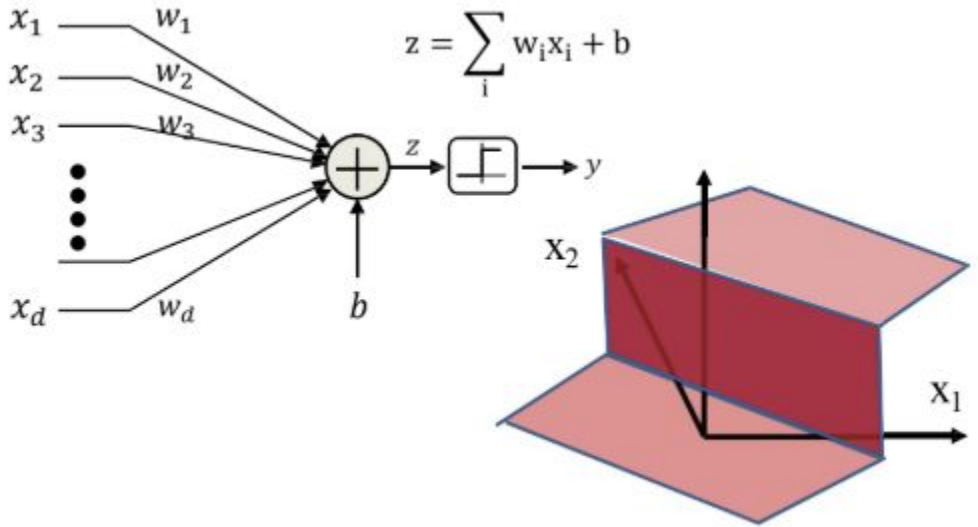
-



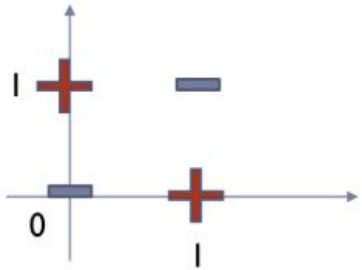


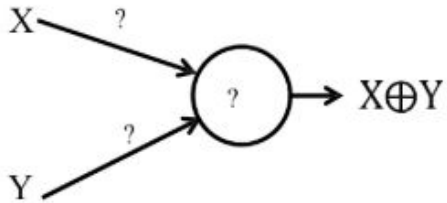
Two-Class Perceptron





What Binary Threshold Neurons Cannot Do





- Cannot compute an XOR

Learning: Binary Perceptron

- Given N training instances $(\boldsymbol{x}^{(1)}, y^{(1)}), (\boldsymbol{x}^{(2)}, y^{(2)}), \ldots, (\boldsymbol{x}^{(N)}, y^{(N)})$
  - $y^{(n)} = +1$ or -1

- Initialize $\boldsymbol{w}$
- Cycle through the training instance
- While more classification errors
  - For $i = 1 \ldots N_{train}$

$$\hat{y}^{(i)} = sign(\boldsymbol{w}^T \boldsymbol{x}^{(i)})$$

    - If $\hat{y}^{(i)} \neq y^{(i)}$

$$\boldsymbol{w} = \boldsymbol{w} + y^{(i)} \boldsymbol{x}^{(i)}$$

- If instance misclassified:
  - If instance is positive class

$$\boldsymbol{w} = \boldsymbol{w} + \boldsymbol{x}$$

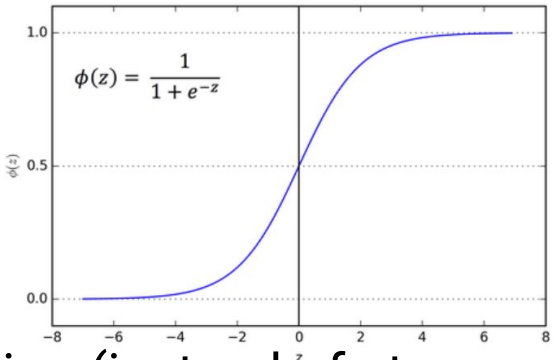
  - If instance is negative class

$$\boldsymbol{w} = \boldsymbol{w} - \boldsymbol{x}$$

How to get probabilistic decisions?

- Perceptron scoring: $z = w \cdot f(x)$
- If $z = w \cdot f(x)$ very positive => want probability going to 1
- If $z = w \cdot f(x)$ very negative => want probability going to 0
- Sigmoid function (logistic function)

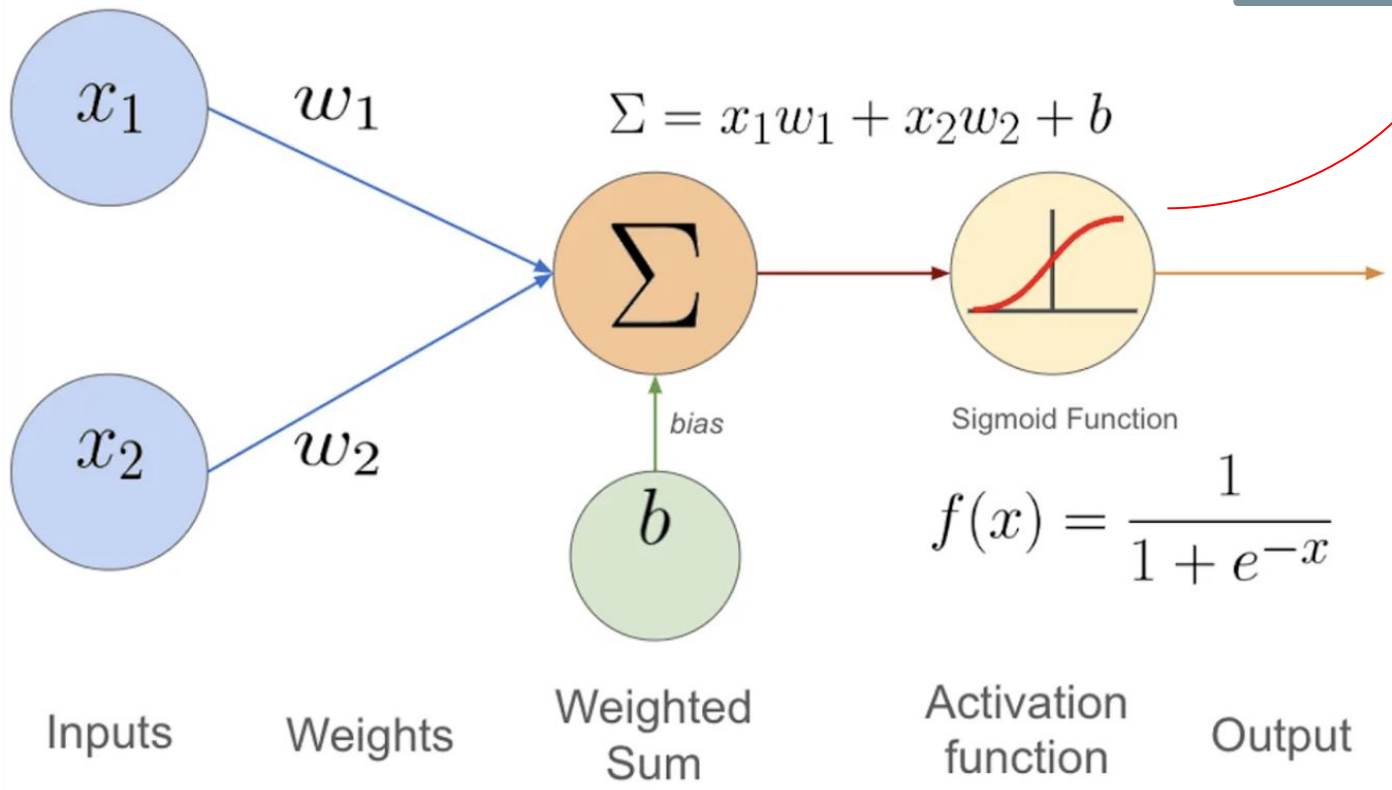
$$\phi(z) = \frac{1}{1 + e^{-z}}$$



- We can use sigmoid as an activation Function (instead of step function)

Two-Class Logistic Regression

Activation Function



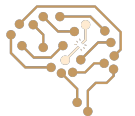
Logistic regression: loss function

Assume we have two classes with labels 0 and 1

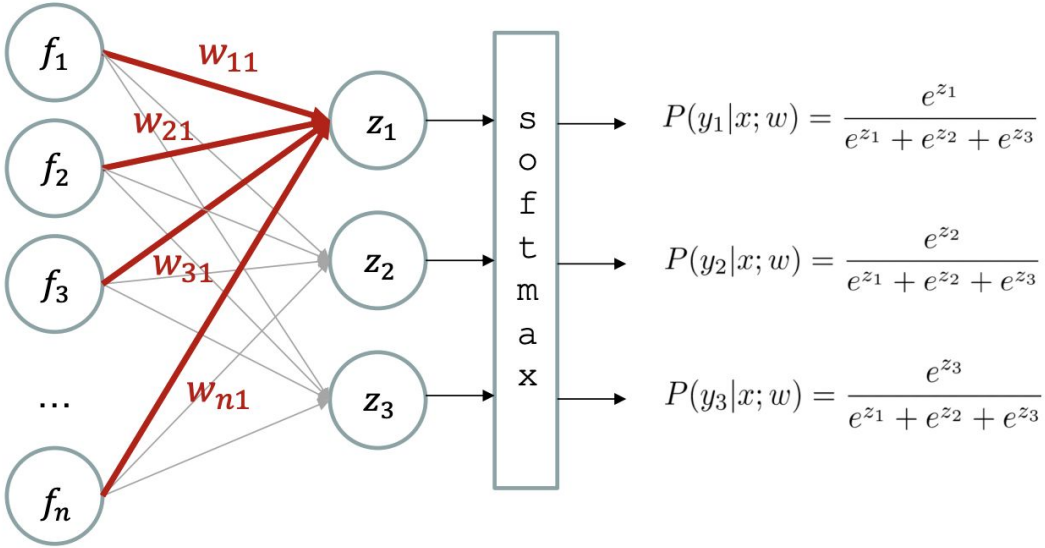
$$\hat{\boldsymbol{w}} = \underset{\boldsymbol{w}}{\operatorname{argmin}} J(\boldsymbol{w})$$

$$J(\boldsymbol{w}) = \sum_{i=1}^{n} -y^{(i)} \log\left(\sigma\left(\boldsymbol{w}^T \boldsymbol{x}^{(i)}\right)\right) - (1 - y^{(i)}) \log\left(1 - \sigma\left(\boldsymbol{w}^T \boldsymbol{x}^{(i)}\right)\right)$$

$J(\boldsymbol{w})$ is convex w.r.t. parameters.



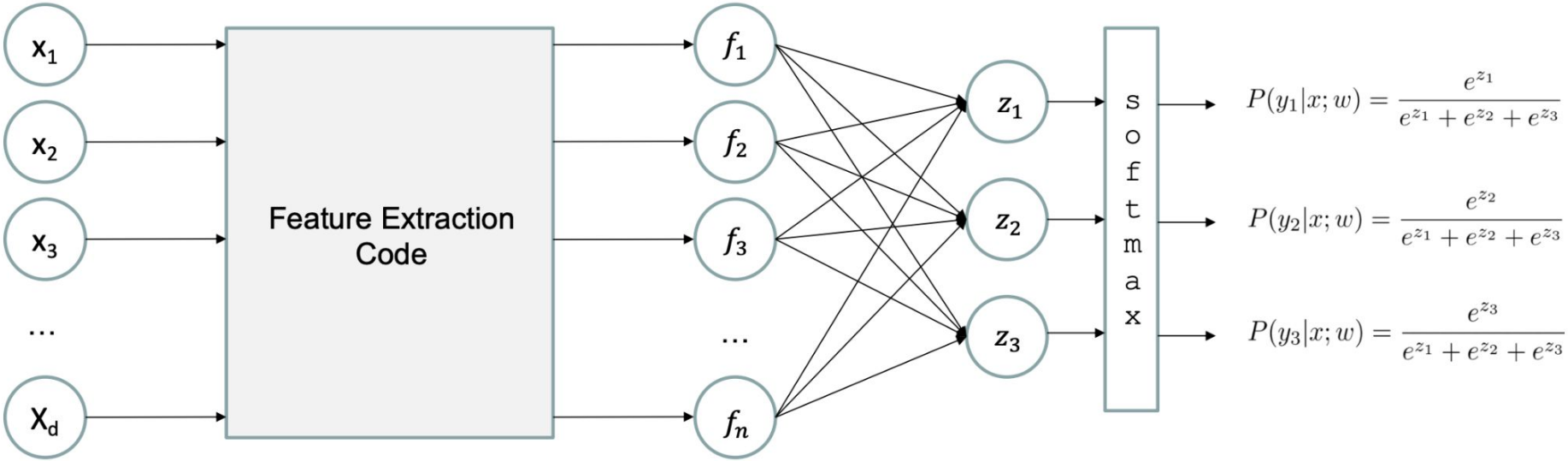
Logistic Regression for 3-way classification



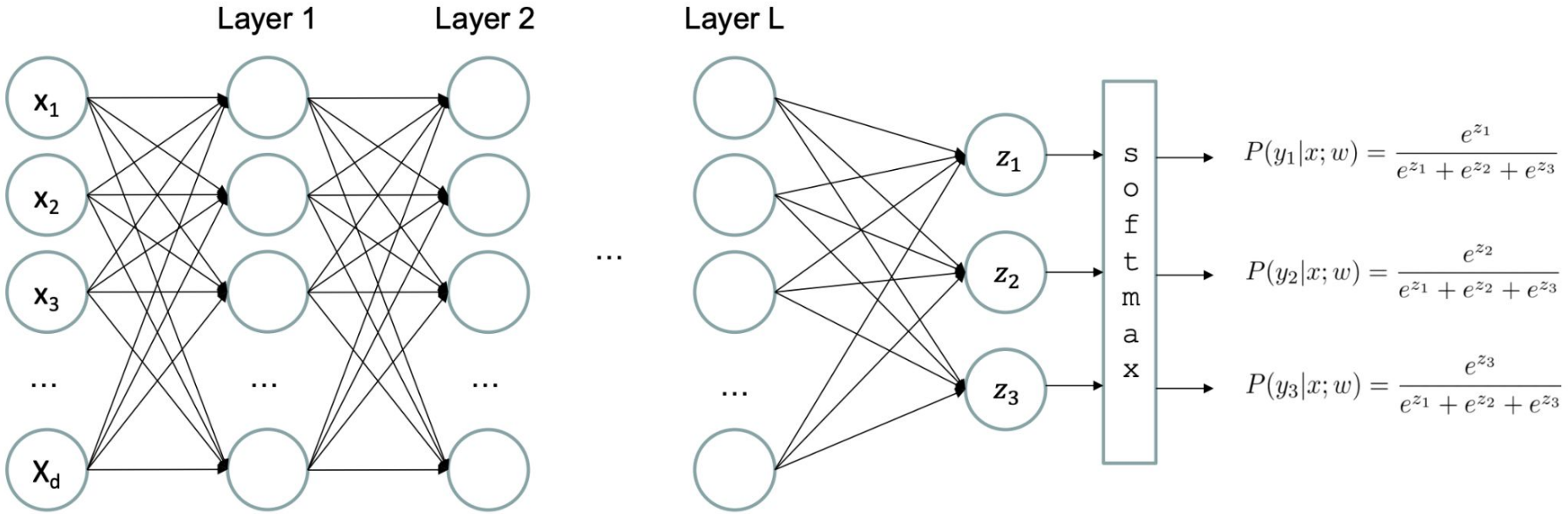
$$z_i = w_i \cdot f$$
$$= \sum_j w_{ji} \cdot f_j$$

Logistic Regression for 3-way classification

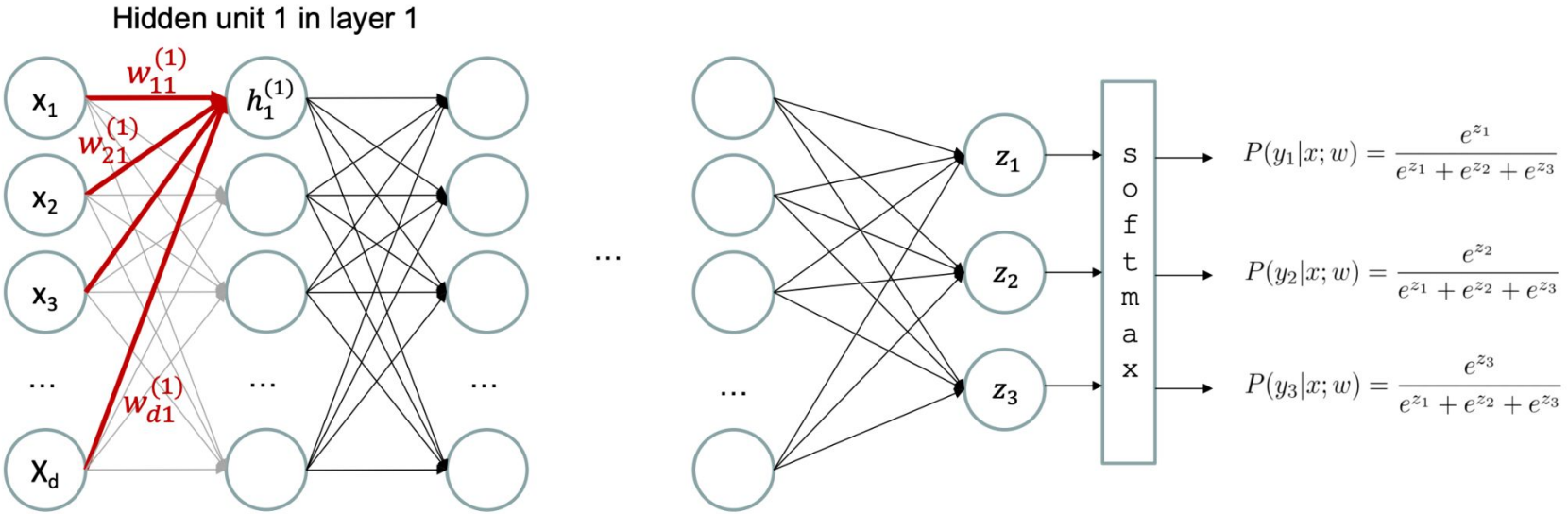
- **Deep Neural Network = Also Learn the Features End-to-End**



Deep Neural Network for 3-way classification



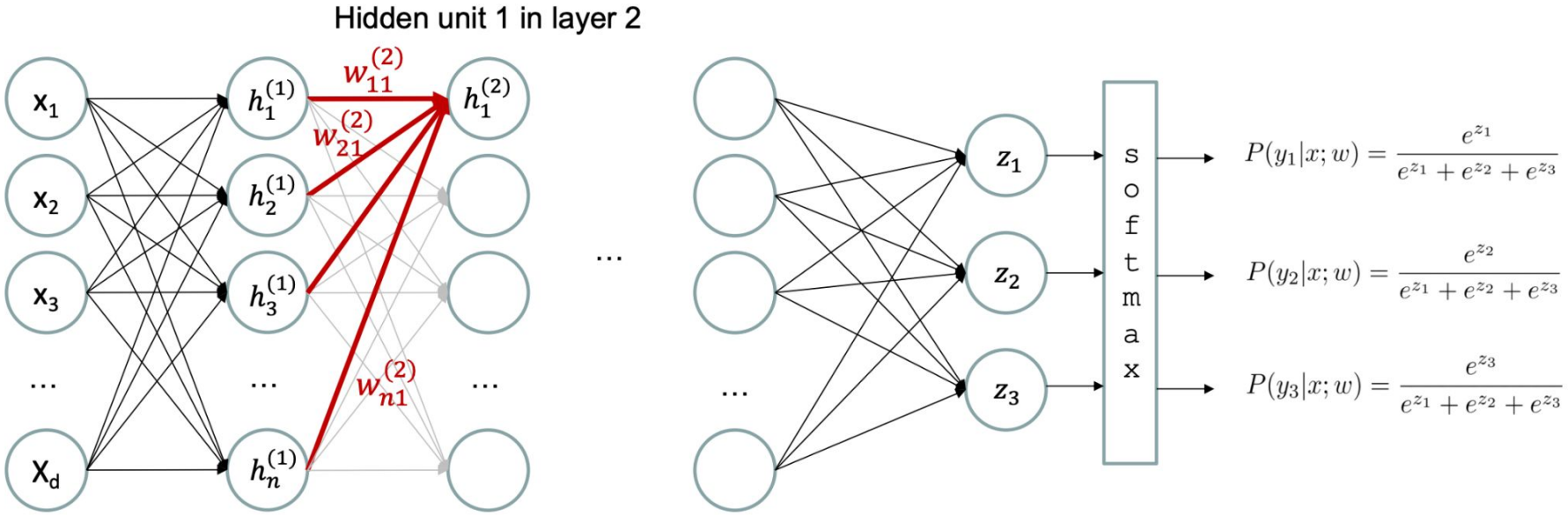
Deep Neural Network for 3-way classification



$$h_1^{(1)} = \phi\left(w_1^{(1)} \cdot x\right) = \phi\left(\sum_j w_{ji}^{(1)} \cdot x_j\right)$$

ϕ = **activation function**

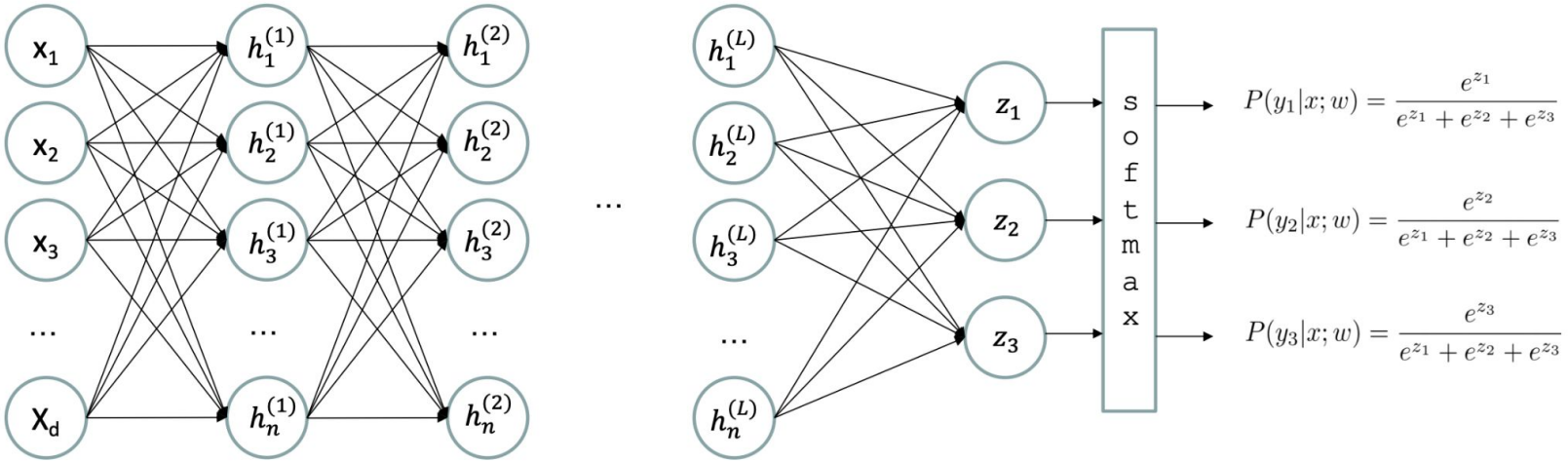
Deep Neural Network for 3-way classification



$$h_1^{(2)} = \phi\left(w_1^{(2)} \cdot h^{(1)}\right) = \phi\left(\sum_j w_{ji}^{(2)} \cdot h_j^{(1)}\right)$$

ϕ = activation function

Deep Neural Network for 3-way classification

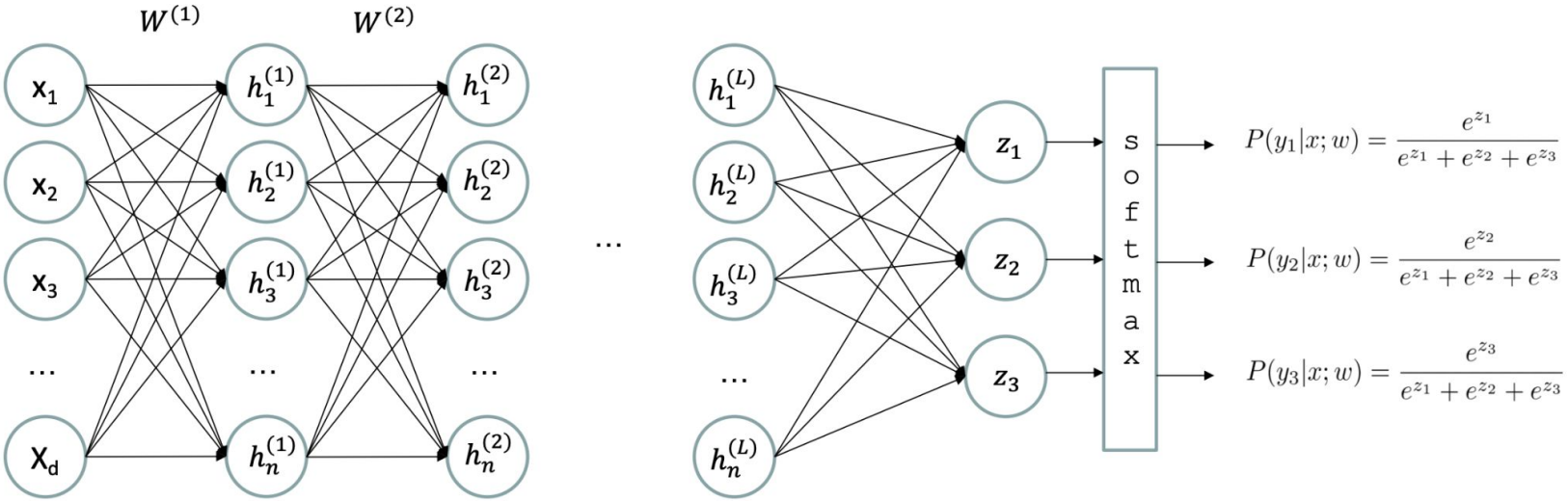


$$h_i^{(l)} = \phi\left(\sum_j w_{ji}^{(l)} \cdot h_j^{(l-1)}\right)$$

ϕ = **activation function**

- Neural network with L layers
- $h^{(l)}$: activations at layer l
- $w^{(l)}$: weights taking activations from layer l-1 to layer l

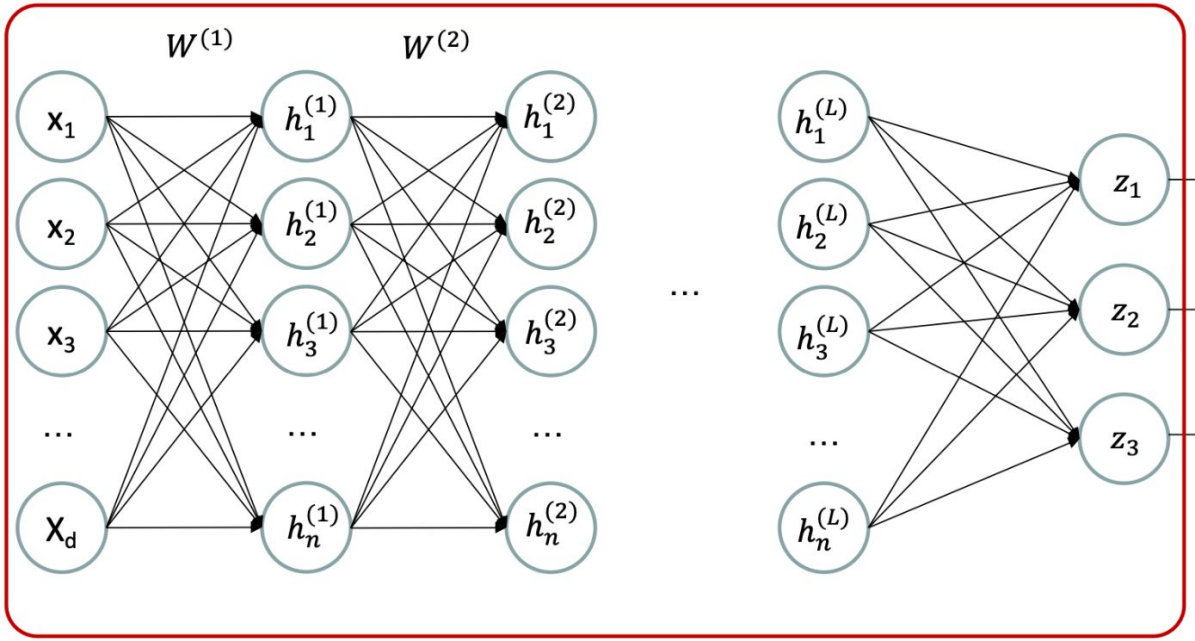
Deep Neural Network for 3-way classification



$$h^{(l)} = \phi(h^{(l-1)} \times W^{(l)})$$

ϕ = activation function

Deep Neural Network for 3-way classification

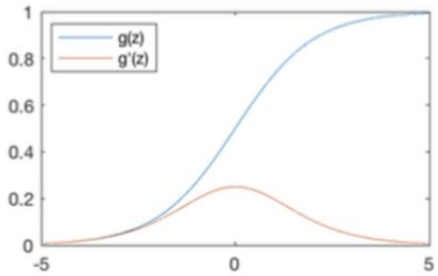


- Sometimes also called *Multi-Layer Perceptron (MLP)* or *Feed-Forward Network (FFN)*

Non Linearity is Important

- Common Activation Functions

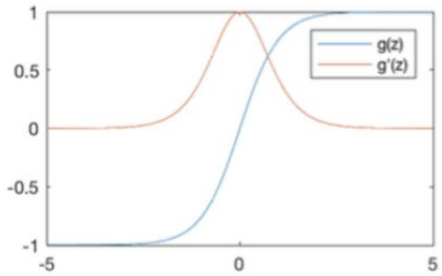
Sigmoid Function



$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

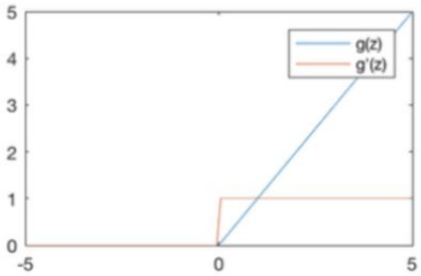
Hyperbolic Tangent



$$g(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$

Rectified Linear Unit (ReLU)



$$g(z) = \max(0, z)$$

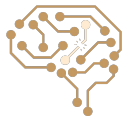
$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

MLP Universal Approximator

- A feed-forward network with a single hidden layer and linear outputs can approximate any continuous function on a compact domain to an arbitrary accuracy
  - under mild assumptions on the activation function
    - e.g., sigmoid activation functions (Cybenko, 1989)
  - when sufficiently large (but finite) number of hidden units is used

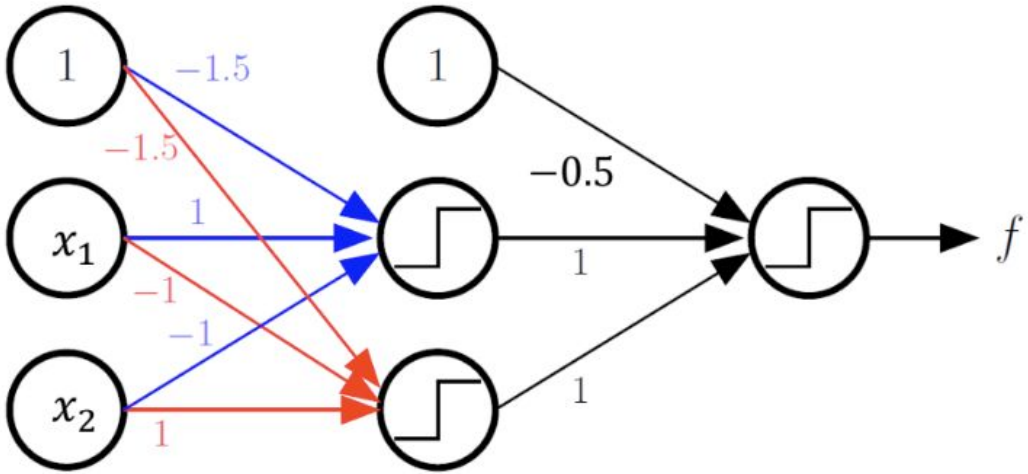
$$F_k(x) = \sum_{j=1}^{M} w_{kj}^{[2]} \phi\left(\sum_{i=0}^{d} w_{ji}^{[1]} x_i\right)$$

- It is of greater theoretical interest than practical
  - the construction of such a network requires the nonlinear activation functions and weight values which are unknown



XOR Example

$$f = XOR(x_1, x_2) = OR\big(AND(x_1, \bar{x}_2), AND(\bar{x}_1, x_2)\big)$$



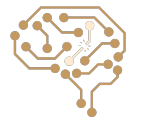
General Boolean Functions

Truth Table

X_1	X_2	X_3	X_4	X_5	Y
0	0	1	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1

Truth table shows all input combinations for which output is 1

$$y = \bar{X}_1\bar{X}_2X_3X_4\bar{X}_5 + \bar{X}_1X_2\bar{X}_3X_4X_5 + \bar{X}_1X_2X_3\bar{X}_4\bar{X}_5 + X_1\bar{X}_2\bar{X}_3\bar{X}_4X_5 + X_1\bar{X}_2X_3X_4X_5 + X_1X_2\bar{X}_3\bar{X}_4X_5$$



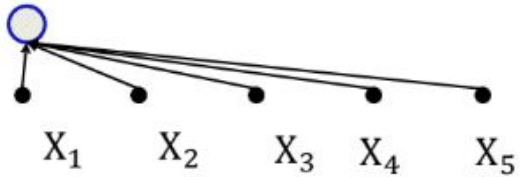
General Boolean Functions (cont.)

Truth Table

X_1	X_2	X_3	X_4	X_5	Y
0	0	1	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1

Truth table shows all input combinations for which output is 1

$$y = \bar{X}_1\bar{X}_2X_3X_4\bar{X}_5 + \bar{X}_1X_2\bar{X}_3X_4X_5 + \bar{X}_1X_2X_3\bar{X}_4\bar{X}_5 + X_1X_2X_3X_4X_5 + X_1\bar{X}_2X_3X_4X_5 + X_1X_2\bar{X}_3\bar{X}_4X_5$$



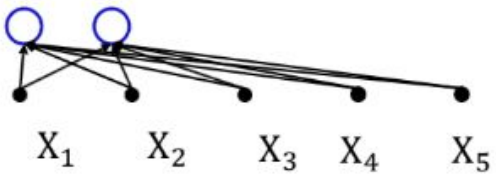
General Boolean Functions (cont.)

Truth Table

X_1	X_2	X_3	X_4	X_5	Y
0	0	1	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1

Truth table shows all input combinations for which output is 1

$$y = \bar{X}_1\bar{X}_2X_3X_4\bar{X}_5 + \bar{X}_1X_2\bar{X}_3X_4X_5 + \bar{X}_1X_2X_3\bar{X}_4\bar{X}_5 + X_1\bar{X}_2\bar{X}_3\bar{X}_4X_5 + X_1X_2X_3X_4X_5 + X_1X_2\bar{X}_3\bar{X}_4X_5$$



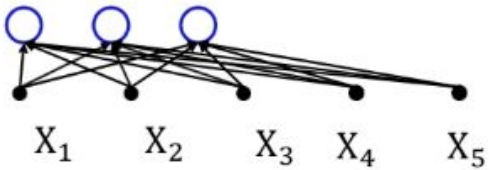
General Boolean Functions (cont.)

Truth Table

X_1	X_2	X_3	X_4	X_5	Y
0	0	1	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1

Truth table shows all input combinations for which output is 1

$$y = \bar{X}_1\bar{X}_2X_3X_4\bar{X}_5 + \bar{X}_1X_2\bar{X}_3X_4X_5 + \bar{X}_1X_2X_3\bar{X}_4\bar{X}_5 + X_1\bar{X}_2\bar{X}_3\bar{X}_4X_5 + X_1\bar{X}_2X_3X_4X_5 + X_1X_2X_3X_4X_5$$



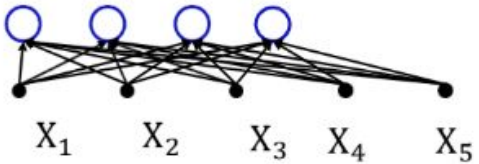
General Boolean Functions (cont.)

Truth Table

X_1	X_2	X_3	X_4	X_5	Y
0	0	1	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1

Truth table shows all input combinations for which output is 1

$$y = \bar{X}_1\bar{X}_2X_3X_4\bar{X}_5 + \bar{X}_1X_2\bar{X}_3X_4X_5 + \bar{X}_1X_2X_3\bar{X}_4\bar{X}_5 + X_1\bar{X}_2\bar{X}_3\bar{X}_4X_5 + X_1\bar{X}_2X_3X_4X_5 + X_1X_2\bar{X}_3\bar{X}_4X_5$$



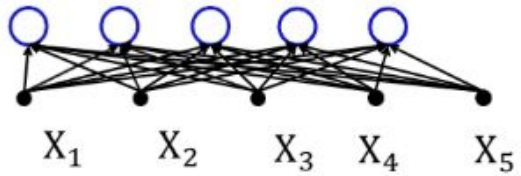
General Boolean Functions (cont.)

Truth Table

X_1	X_2	X_3	X_4	X_5	Y
0	0	1	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1

Truth table shows all input combinations for which output is 1

$$y = \bar{X}_1\bar{X}_2X_3X_4\bar{X}_5 + \bar{X}_1X_2\bar{X}_3X_4X_5 + \bar{X}_1X_2X_3\bar{X}_4\bar{X}_5 + X_1\bar{X}_2\bar{X}_3\bar{X}_4X_5 + X_1\bar{X}_2X_3X_4X_5 + X_1X_2\bar{X}_3\bar{X}_4X_5$$



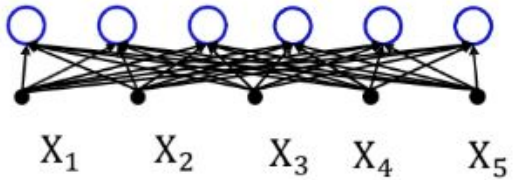
General Boolean Functions (cont.)

Truth Table

X_1	X_2	X_3	X_4	X_5	Y
0	0	1	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1

Truth table shows all input combinations for which output is 1

$$y = \bar{X}_1\bar{X}_2X_3X_4\bar{X}_5 + \bar{X}_1X_2\bar{X}_3X_4X_5 + \bar{X}_1X_2X_3\bar{X}_4\bar{X}_5 + X_1\bar{X}_2\bar{X}_3\bar{X}_4X_5 + X_1\bar{X}_2X_3X_4X_5 + X_1X_2\bar{X}_3\bar{X}_4X_5$$



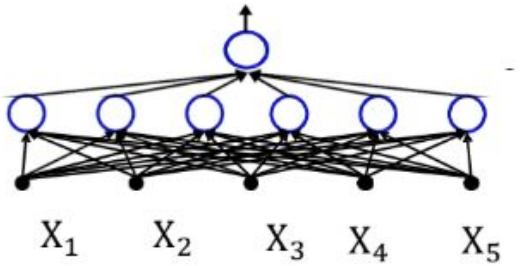
General Boolean Functions (cont.)

Truth Table

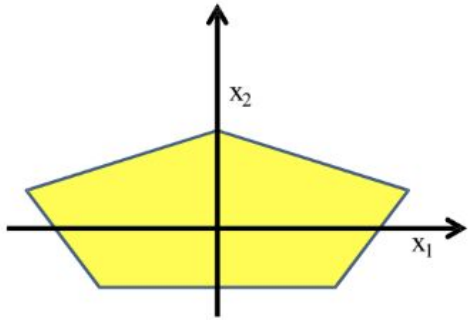
X_1	X_2	X_3	X_4	X_5	Y
0	0	1	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
1	0	0	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1

Truth table shows all input combinations for which output is 1

$$y = \bar{X}_1\bar{X}_2X_3X_4\bar{X}_5 + \bar{X}_1X_2\bar{X}_3X_4X_5 + \bar{X}_1X_2X_3\bar{X}_4\bar{X}_5 + X_1\bar{X}_2\bar{X}_3\bar{X}_4X_5 + X_1\bar{X}_2X_3X_4X_5 + X_1X_2\bar{X}_3\bar{X}_4X_5$$



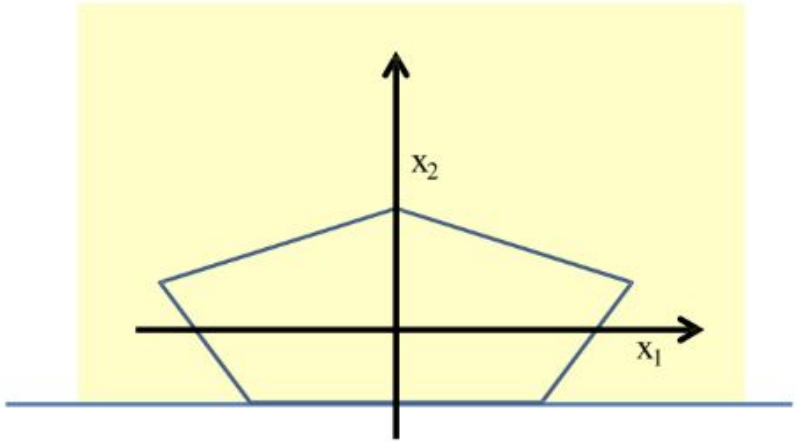
General Functions

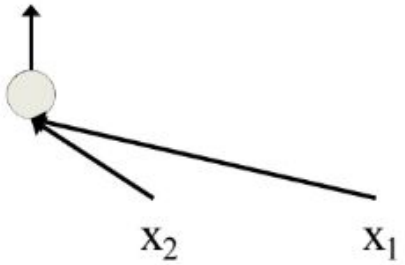


Can now be composed into "networks" to compute arbitrary classification "boundaries'

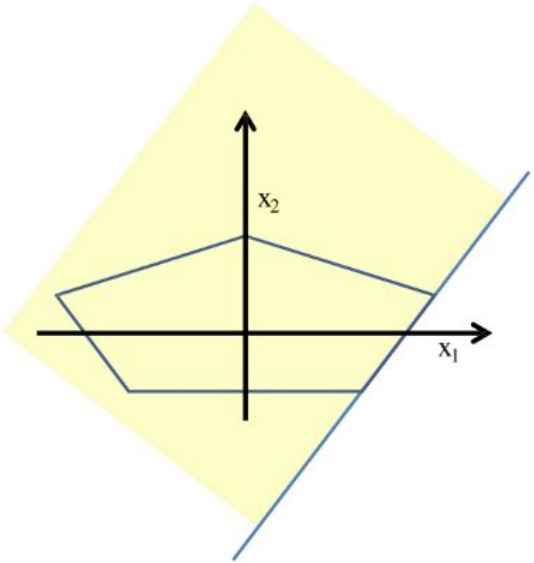
- Build a network of units with a single output that fires if the input is in the coloured area

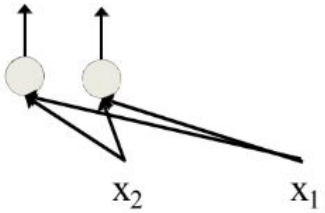
General Functions (cont.)



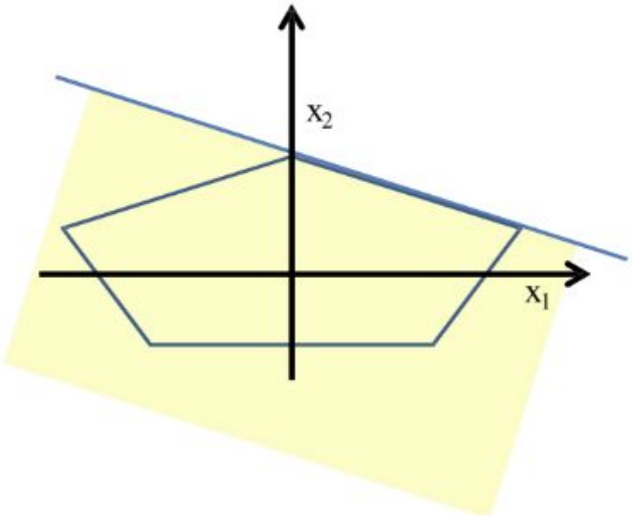


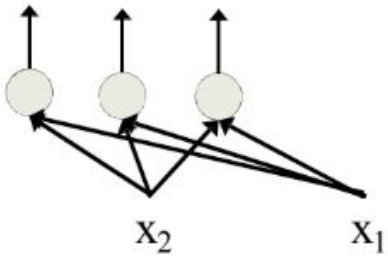
General Functions (cont.)



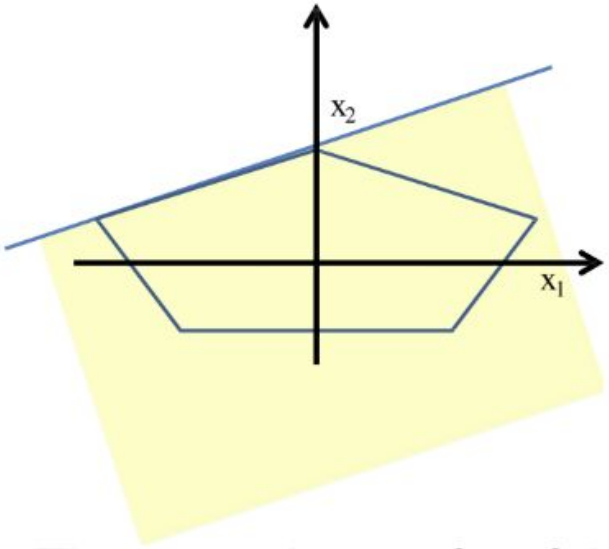


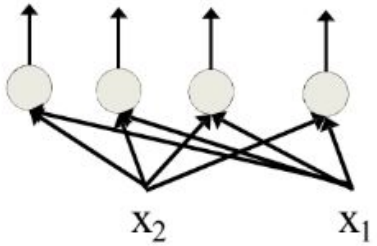
General Functions (cont.)



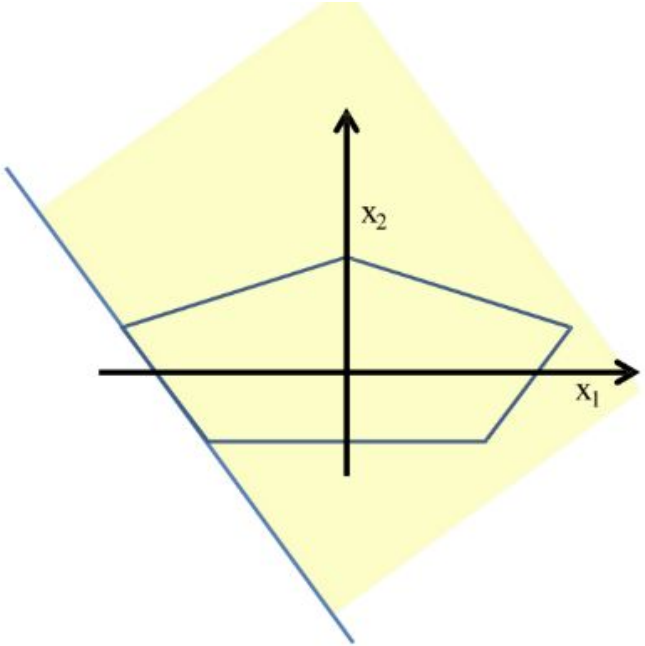


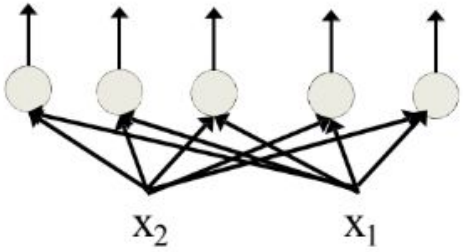
General Functions (cont.)



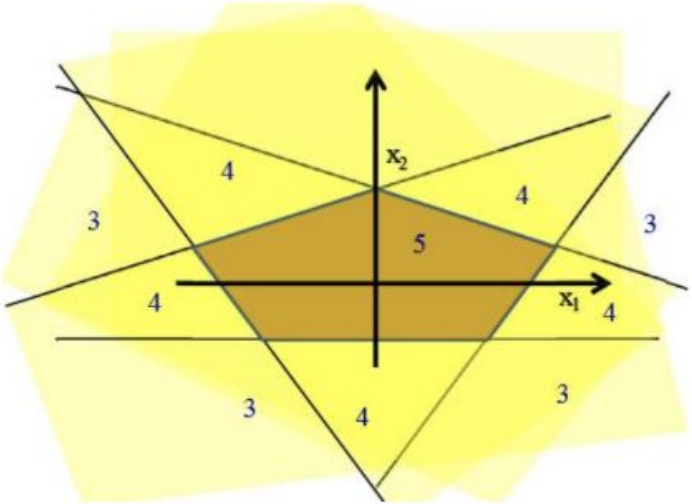


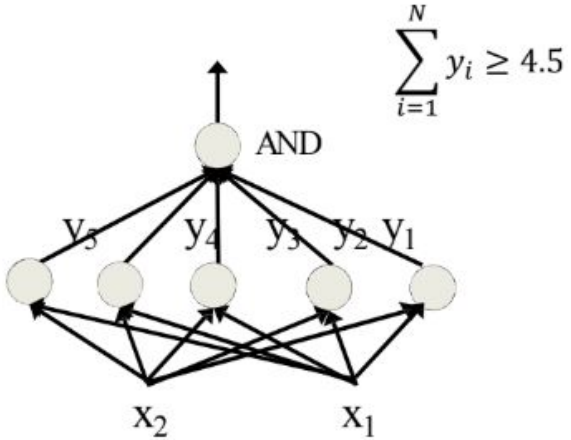
General Functions (cont.)



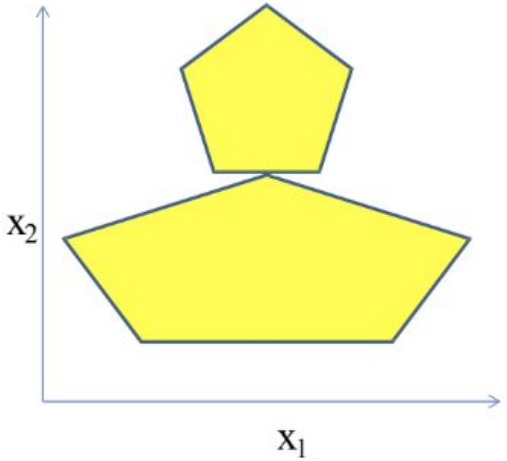


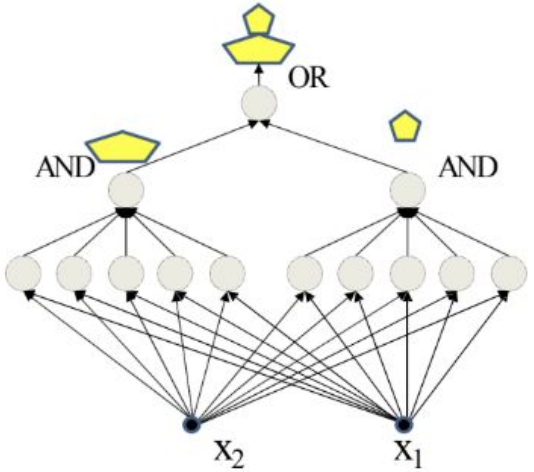
General Functions (cont.)



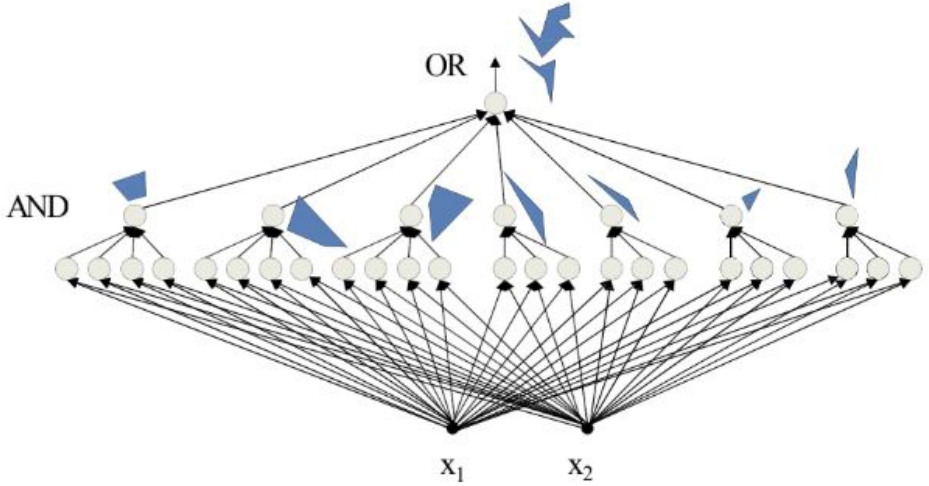


More Complex Decision Boundaries





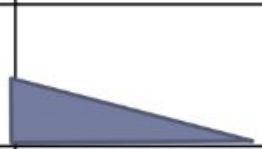
More Complex Decision Boundaries (cont.)



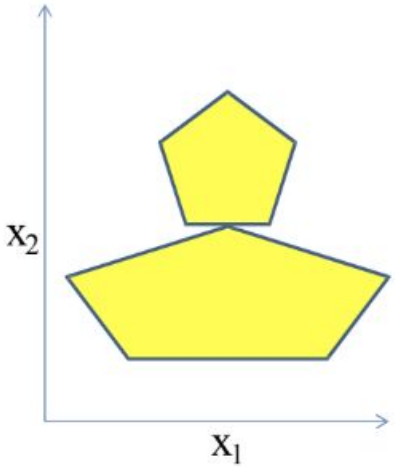
MLP with Different Number of Layers

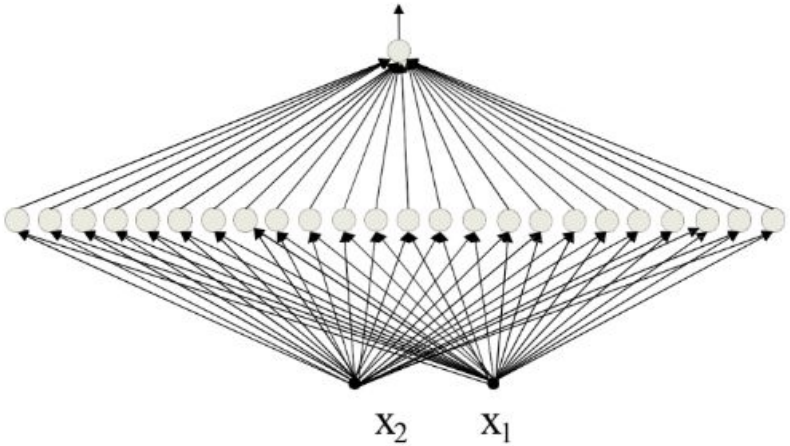
MLP with unit step activation function

Decision region found by an output unit.

Structure	Type of Decision Regions	Interpretation	Example of region
Single Layer (no hidden layer)	Half space	Region found by a hyper-plane	
Two Layer (one hidden layer)	Polyhedral (open or closed) region	Intersection of half spaces	
Three Layer (two hidden layers)	Arbitrary regions	Union of polyhedrals	

Exercise: Compose This with One Hidden Layer





Expressive Power of MLPs

- MLPs are universal Boolean function
- MLPs are universal classifiers
- MLPs are universal function approximators

- An MLP with two (or even one) hidden layers can approximate anything to arbitrary precision
  - But could be exponentially or even infinitely wide in its inputs size

Learning Problem in Summary

- Given: the architecture of the network
- Training data: A set of input-output pairs

$$(\boldsymbol{x}^{(1)}, \boldsymbol{y}^{(1)}), (\boldsymbol{x}^{(2)}, \boldsymbol{y}^{(2)}), \ldots, (\boldsymbol{x}^{(N)}, \boldsymbol{y}^{(N)})$$

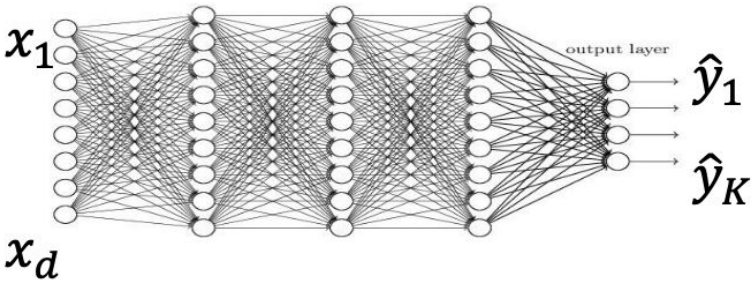
- We want to find the function g (model) on the input to get the output
- We consider a neural network as a parametric function $g(\boldsymbol{x};\boldsymbol{W})$
- We need a loss function to show how penalizes the obtained output $g(\boldsymbol{x};\boldsymbol{W})$ when the desired output is $\boldsymbol{y}$

$$\frac{1}{N}\sum_{n=1}^{N} loss\left(g\left(\boldsymbol{x}^{(n)};\boldsymbol{W}\right), \boldsymbol{y}^{(n)}\right)$$

Choosing Cost Function: Examples

> Regression problem (multiple outputs)

SSE Loss



$$E = \sum_{n=1}^{N} E_n$$

$$E_n = \sum_{k=1}^{K} \left(\hat{y}_k^{(n)} - y_k^{(n)}\right)^2$$

> Classification problem

$$loss_n = \sum_{k=1}^{K} -y_k^{(n)} \log \hat{y}_k^{(n)}$$

Cross Entropy Loss

Output is found by a softmax layer $\hat{y}_k = \frac{e^{z_k}}{\sum_{k=1}^{K} e^{z_k}}$

Deep Neural Network Training

- Do the maximum likelihood estimation is a good Idea …
- Find a set of parameters which results in the best performance in training data
- In general, cannot always take derivative and set to 0
- Use numerical optimization!

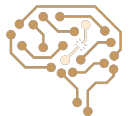
- **GRADIENT DESCENT!**
- Hill climbing in continuous space

The Main Issue of Using Gradient Descent for NNs

- The loss function (L) itself is a function of model g
- The model g itself is a complex function of weights (parameters) $W^{(1)}$, $W^{(2)}$, …, $W^{(L)}$
- We want the gradient of the loss function w.r.t to the parameters

Gradient in n Dimensions

$$\nabla L = \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \cdots \\ \frac{\partial L}{\partial w_n} \end{bmatrix}$$



Optimization Procedure: Gradient Descent

Init w

for iter = 1, 2, …

$$w \leftarrow w - \alpha \cdot \nabla L(w)$$

L is the loss function

α: learning rate

Univariate Chain Rule

$$\frac{\mathrm{d}}{\mathrm{d}t} f(x(t)) = \frac{\mathrm{d}f}{\mathrm{d}x}\frac{\mathrm{d}x}{\mathrm{d}t}.$$

- Example:

Computing the loss:

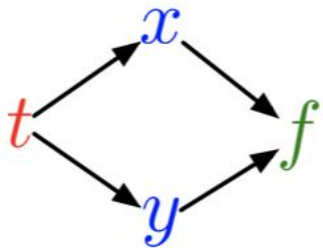
$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

Computing the derivatives:

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}y} = y - t$$
$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}z} = \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}y}\sigma'(z)$$
$$\frac{\partial\mathcal{L}}{\partial w} = \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}z}x$$
$$\frac{\partial\mathcal{L}}{\partial b} = \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}z}$$

Multivariate Chain Rule

$$\frac{\mathrm{d}}{\mathrm{d}t} f(x(t), y(t)) = \frac{\partial f}{\partial x}\frac{\mathrm{d}x}{\mathrm{d}t} + \frac{\partial f}{\partial y}\frac{\mathrm{d}y}{\mathrm{d}t}$$



Example:

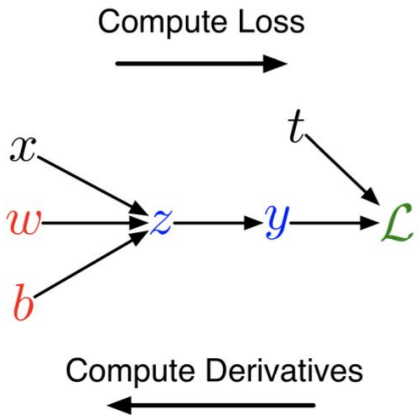
$$\begin{aligned} f(x, y) &= y + e^{xy} \\ x(t) &= \cos t \\ y(t) &= t^2 \end{aligned}$$

Plug in to Chain Rule:

$$\begin{aligned} \frac{\mathrm{d}f}{\mathrm{d}t} &= \frac{\partial f}{\partial x}\frac{\mathrm{d}x}{\mathrm{d}t} + \frac{\partial f}{\partial y}\frac{\mathrm{d}y}{\mathrm{d}t} \\ &= (ye^{xy}) \cdot (-\sin t) + (1 + xe^{xy}) \cdot 2t \end{aligned}$$

Computation Graph

- We can diagram out the computations using a computation graph.
- The nodes represent all the inputs and computed quantities, and the edges represent which nodes are computed directly as a function of other nodes.



Backpropagation Algorithm

- Let v_1, …, v_N be a topological ordering of the computation graph (i.e. parents come before children.)
- v_N denotes the variable we are trying to compute derivatives of (e.g. loss)

forward pass

For $i = 1, \ldots, N$

Compute v_i as a function of $\mathrm{Pa}(v_i)$

backward pass

$\overline{v_N} = 1$

For $i = N - 1, \ldots, 1$

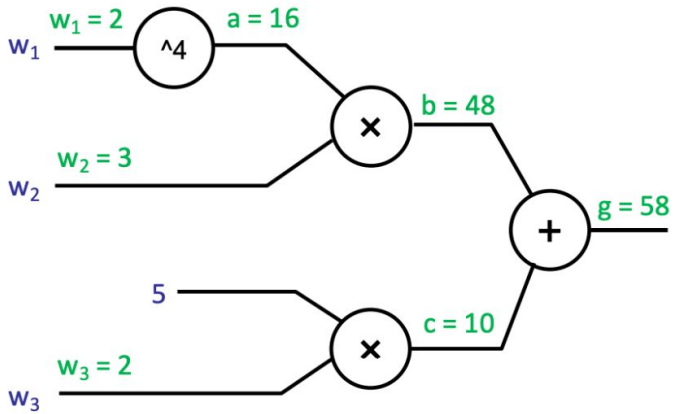
$\overline{v_i} = \sum_{j \in \mathrm{Ch}(v_i)} \overline{v_j} \frac{\partial v_j}{\partial v_i}$

Backpropagation Example 1

Gradient of $g(w_1, w_2, w_3) = w_1^4 w_2 + 5w_3$ at $w_1 = 2$, $w_2 = 3$, $w_3 = 2$

Think of *g* as a composition of many functions

- Then, we can use the chain rule to compute the gradient



g = b + c

$$\frac{\partial g}{\partial b} = 1, \frac{\partial g}{\partial c} = 1$$

b = a × w_2

$$\frac{\partial g}{\partial a} = \frac{\partial g}{\partial b}\frac{\partial b}{\partial a} = 1 \cdot w_2 = 3 \qquad \frac{\partial g}{\partial w_2} = \frac{\partial g}{\partial b}\frac{\partial b}{\partial w_2} = 1 \cdot a = 16$$

a = w_1^4

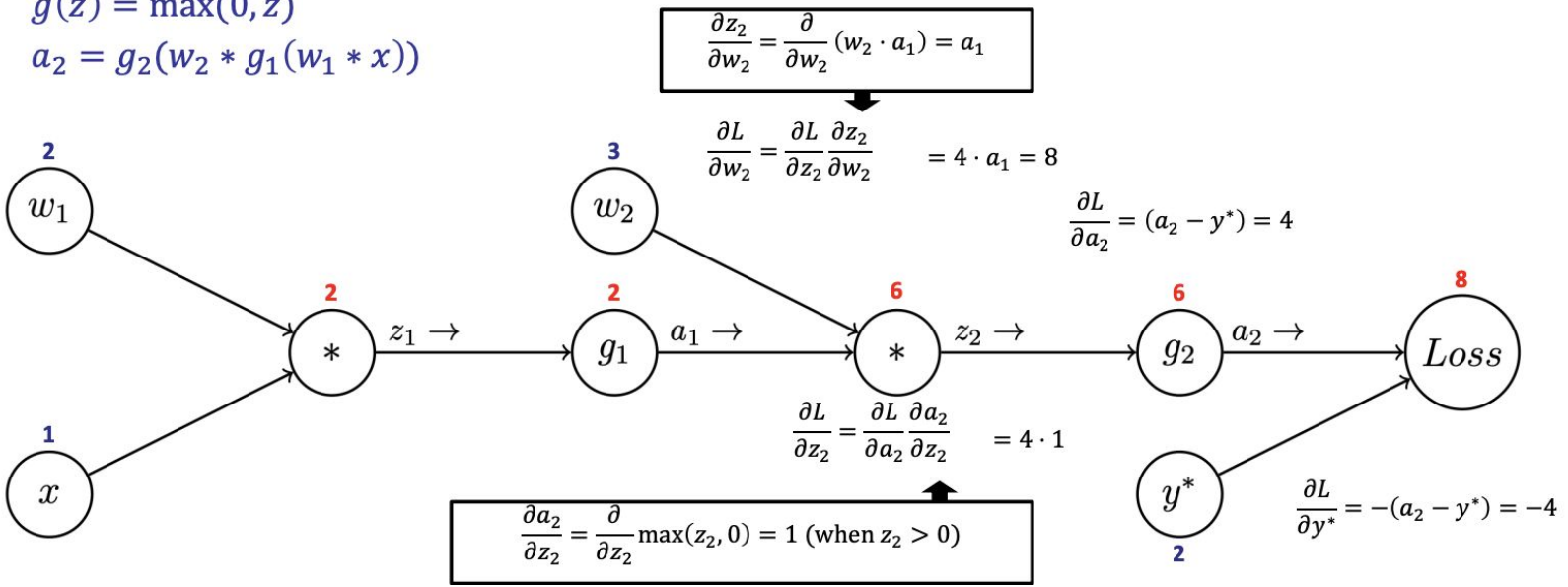
$$\frac{\partial g}{\partial w_1} = \frac{\partial g}{\partial a}\frac{\partial a}{\partial w_1} = 3 \cdot 4w_1^3 = 96$$

c = 5w_1

$$\frac{\partial g}{\partial w_3} = \frac{\partial g}{\partial c}\frac{\partial c}{\partial w_3} = 1 \cdot 5 = 5$$

Backpropagation Example 2

- Build a computation graph and apply chain rule: $f(x) = g(h(x))$ $f'(x) = h'(x) \cdot g'(h(x))$
- Example: neural network with quadratic loss: $L(a_2, y^*) = \frac{1}{2}(a_2 - y^*)^2$ and ReLU activations $g(z) = \max(0, z)$
- $a_2 = g_2(w_2 * g_1(w_1 * x))$

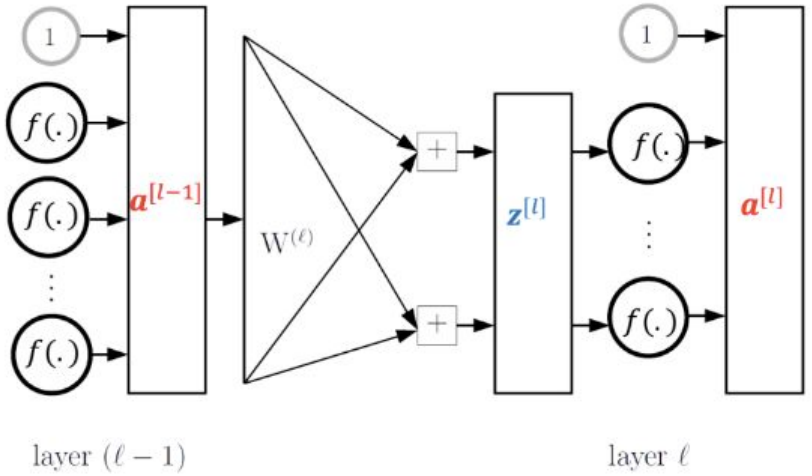


Main Goal of Backpropagation

- How to compute $\frac{d\ loss(\boldsymbol{o}, \boldsymbol{y})}{dw_{ji}^{[k]}}$

Backpropagation: Notation

- $\boldsymbol{a}^{[0]} \leftarrow Input$
- $output \leftarrow \boldsymbol{a}^{[L]}$



Backpropagation: Last Layer Gradient

For squared error loss:

$$loss = \frac{1}{2}\sum_j (o_j - y_j)^2$$

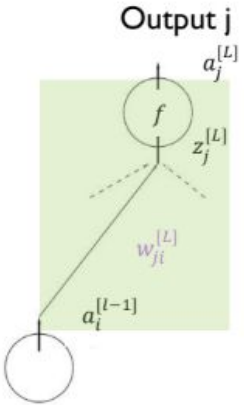
$$o_j = a_j^{[L]}$$

$$\frac{\partial loss}{\partial a_j^{[L]}} = (a_j^{[L]} - y_j)$$

$$\frac{\partial loss}{\partial w_{ji}^{[L]}} = ?$$

$$a_i^{[L]} = f\left(z_i^{[L]}\right)$$

$$z_j^{[L]} = \sum_{i=0}^{M} w_{ji}^{[L]} a_i^{[L-1]}$$



$\frac{\partial loss}{\partial a_j^{[L]}}$

$\frac{\partial loss}{\partial w_{ji}^{[L]}}$

Backpropagation: Last layer gradient

Activation output of neuron number j in layer L

Pre activation value (before applying the activation function) of neuron number j in layer L

For squared error loss:

$$loss = \frac{1}{2}\sum_j (\hat{y}_j - y_j)^2$$

$$\hat{y}_j = a_j^{[L]}$$

$$\frac{\partial loss}{\partial a_j^{[L]}} = (a_j^{[L]} - y_j)$$

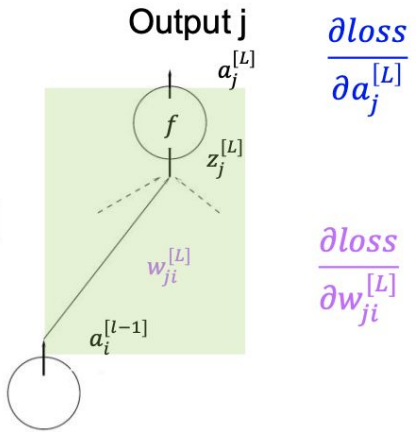
$$a_j^{[L]} = f\left(z_j^{[L]}\right)$$

$$z_j^{[L]} = \sum_{i=0}^{M} w_{ji}^{[L]} a_i^{[L-1]}$$

f is the activation function

$$\frac{\partial loss}{\partial w_{ji}^{[L]}} = \frac{\partial loss}{\partial a_j^{[L]}} \frac{\partial a_j^{[L]}}{\partial w_{ji}^{[L]}}$$

$$\frac{\partial a^{[L]}}{\partial w_{ji}^{[L]}} = f'\left(z_j^{[L]}\right) \frac{\partial z_j^{[L]}}{\partial w_{ji}^{[L]}} = f'\left(z_j^{[L]}\right) a_i^{[L-1]}$$



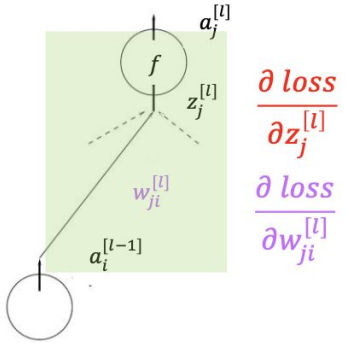
$$\frac{\partial loss}{\partial w_{ji}^{[L]}} = \frac{\partial loss}{\partial a_j^{[L]}} f'\left(z_j^{[L]}\right) a_i^{[L-1]}$$

Previous Layers Gradients

$$\frac{\partial\ loss}{\partial w_{ji}^{[l]}} = \frac{\partial\ loss}{\partial z_j^{[l]}} \frac{\partial z_j^{[l]}}{\partial w_{ji}^{[l]}} = \frac{\partial\ loss}{\partial z_j^{[l]}} a_i^{[l-1]}$$

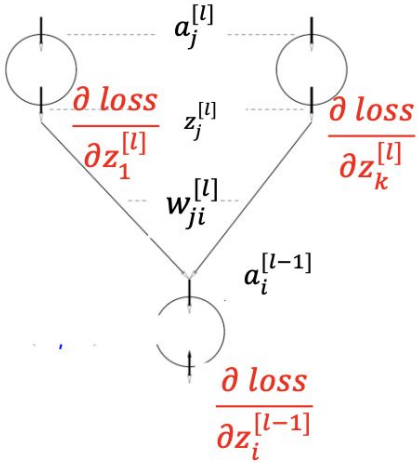
$$a_j^{[l]} = f\left(z_j^{[l]}\right)$$

$$z_j^{[l]} = \sum_{i=0}^{M} w_{ji}^{[l]} a_i^{[l-1]}$$



$$\frac{\partial\ loss}{\partial z_i^{[l-1]}} = \frac{\partial a_i^{[l-1]}}{\partial z_i^{[l-1]}} \sum_{j=1}^{d^{[l]}} \frac{\partial\ loss}{\partial z_j^{[l]}} \times \frac{\partial z_j^{[l]}}{\partial a_i^{[l-1]}}$$

$$= f'\left(z_i^{[l-1]}\right) \sum_{j=1}^{d^{[l]}} \frac{\partial\ loss}{\partial z_j^{[l]}} \times w_{ji}^{[l]}$$



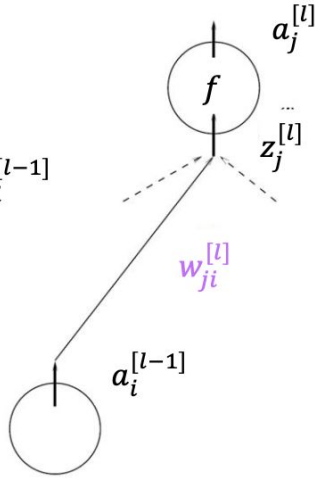
Backpropagation

$$\frac{\partial\, loss}{\partial w_{ji}^{[l]}} = \boxed{\frac{\partial\, loss}{\partial z_j^{[l]}}} \times \frac{\partial z_j^{[l]}}{\partial w_{ji}^{[l]}}$$

$$= \boxed{\delta_j^{[l]}} \times a_i^{[l-1]}$$

$$a_j^{[l]} = f\left(z_j^{[l]}\right)$$

$$z_j^{[l]} = \sum_{i=0}^{M} w_{ji}^{[l]} a_i^{[l-1]}$$

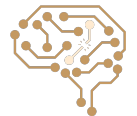


$\delta_j^{[l]} = \frac{\partial\, loss}{\partial z_j^{[l]}}$ is the sensitivity of the loss to $z_j^{[l]}$

Sensitivity vectors can be obtained by running a backward process in the network architecture (hence the name backpropagation.)

We will compute $\boldsymbol{\delta}^{[l-1]}$ from $\boldsymbol{\delta}^{[l]}$:

$$\delta_i^{[l-1]} = f'\left(z_i^{[l-1]}\right) \sum_{j=1}^{d^{[l]}} \delta_j^{[l]} \times w_{ji}^{[l]}$$



Backpropagation Algorithm (Gradient Descent)

Initialize all weights to small random numbers.

While not satisfied

For each training example do:

1. Feed forward the training example to the network and compute the outputs of all units in forward step (z and a) and the loss
2. For each unit find its δ in the backward step
3. Update each network weight $w_{ij}^{[l]}$ as $w_{ij}^{[l]} \leftarrow w_{ij}^{[l]} - \eta \frac{\partial\, loss}{\partial w_{ij}^{[l]}}$ where $\frac{\partial\, loss}{\partial w_{ij}^{[l]}}$

$$= \delta_j^{[l]} \times a_i^{[l-1]}$$

Next Session: Convolutional Neural Networks (CNNs)